Anti-Money Laundering (AML) Detection

Machine Learning, FastAPI, GitHub Actions, Azure Machine Learning and MLflow

Capstone Project Report

Track chosen: Anti-Money Laundering (SAML-D transaction dataset)

Name: Rahul Kumar Dubey

Email Id- Rahul.3.Dubey@coforge.com

E-Code- 81010076

Guide / Institution: Mr. Amit Mishra

Contents

- 1. Abstract and problem statement
- 2. Dataset description and preprocessing
- 3. Exploratory data analysis
- 4. Feature engineering, imbalance handling and validation strategy
- 5. Model building, tuning, evaluation and selection
- 6. Model serialisation (.pkl) and validation
- 7. FastAPI design, testing and CI pipeline
- 8. Azure MLflow implementation: tracking, registry, deployment, monitoring
- 9. Results, limitations, ethical considerations and future work
- 10. References
- Appendices A to C

1. Abstract and problem statement

Abstract

This project builds a complete machine-learning system for detecting suspicious transactions in the SAML-D anti-money-laundering dataset. Three scikit-learn models (Logistic Regression, Decision Tree, Random Forest) were compared under several class-imbalance strategies using a chronological train/validation/test split. The selected model, Random Forest, reached a PR-AUC of 0.797 and ROC-AUC of 0.883 on the held-out test window, with precision 0.847, recall 0.700 and F1 0.767 at a threshold chosen on the validation window. The model is served through a FastAPI service, tested automatically and prepared for deployment as an Azure ML managed online endpoint with MLflow tracking.

Problem statement

Financial institutions must detect and report suspicious activity. Missed cases bring regulatory penalties, while excessive false alerts overload investigators. The task is to classify each transaction as normal or laundering when only 0.1039% of transactions are laundering, so accuracy alone is misleading and model selection uses precision, recall, F1 and precision-recall AUC.

Track chosen: Anti-Money Laundering detection (SAML-D). The label column is Is_laundering.

2. Dataset description and preprocessing

The raw file contains 9,504,852 transactions. After merging Date and Time into one Timestamp column the working table has 11 columns, covering 2022-10-07 10:35:19 to 2023-08-23 10:57:12, with 292,715 distinct sender and 652,266 receiver accounts. Memory footprint after typed loading is 332.7 MB.

Table 1. Schema, data types and missing values

Column	Type	Missing
Sender_account	int64	0
Receiver_account	int64	0
Amount	float32	0
Payment_currency	category	0
Received_currency	category	0
Sender_bank_location	category	0
Receiver_bank_location	category	0
Payment_type	category	0
Is_laundering	int8	0
Laundering_type	category	0
Timestamp	datetime64[us]	0

Table 2. Summary statistics of Amount

Statistic	Amount
count	9,504,852.0000
mean	8,762.9676
std	25,614.9529
min	3.7300
25%	2,143.6874
50%	6,113.7202
75%	10,458.4624
max	12,618,498.0000

Table 3. Target distribution and class imbalance

Target statistic	Value
Positives (laundering)	9,873
Negatives (normal)	9,494,979
Positive rate (%)	0.1039
Imbalance ratio (negative : positive)	961.7000

Cleaning decisions

- Exact duplicate rows removed: 0.
- Rows removed for missing required fields: 0.
- Rows removed for non-positive amounts: 0.
- Self-transfers found: 0.
- Rows kept: 9,504,852 of 9,504,852.
- Amount outliers above the IQR fence (22,930.62): 429,848 rows, with laundering rate 0.208% against 0.099% below the fence. They are retained as signal and compressed with a log transform.

Leakage control: the column Laundering_type is a label-derived typology that fully determines the target, so it is never used as a feature. It is used only for error analysis.

3. Exploratory data analysis

All EDA was computed on the training window only; validation and test were not inspected for modelling decisions.

Table 4. Chronological split

Split	Rows	Positives	Positive rate	Start	End

train	6,653,396	6,764	0.1017%	2022-10-07	2023-05-18
valid	1,425,728	1,412	0.0990%	2023-05-18	2023-07-05
test	1,425,728	1,697	0.1190%	2023-07-05	2023-08-23

Leakage audit: Laundering_type has 28 values; 0 appear in both classes, confirming it cannot be used as a feature.

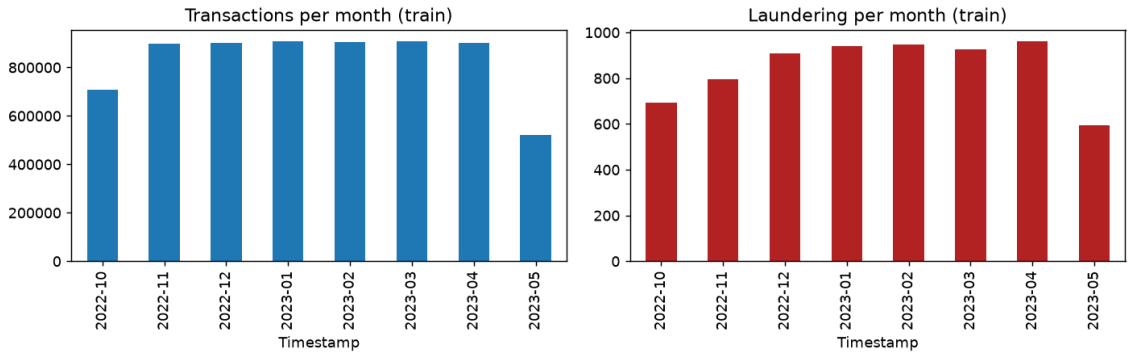


Figure 1. Monthly transactions and laundering cases (training window)

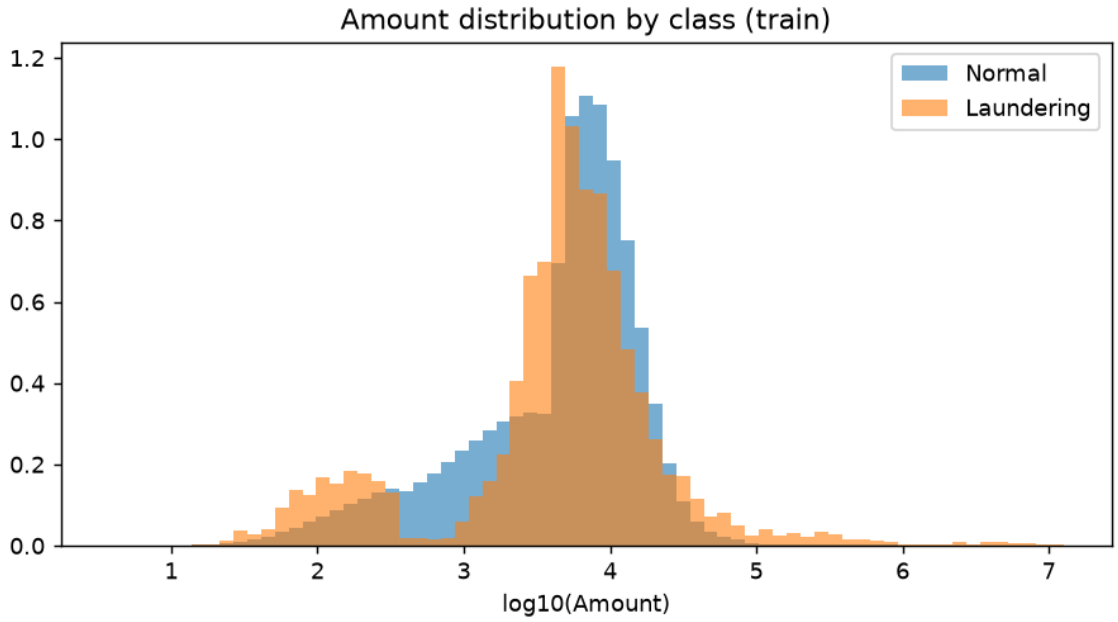


Figure 2. Amount distribution by class (log scale)

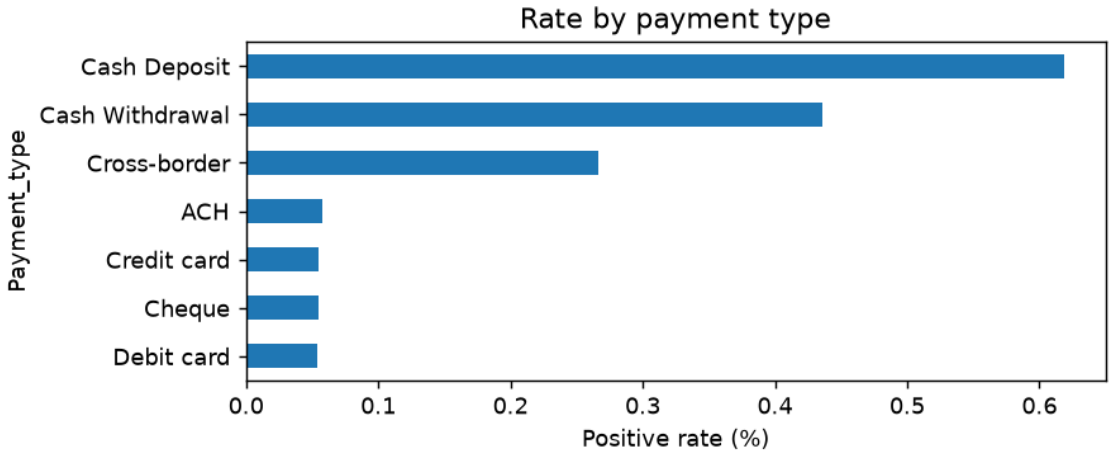


Figure 3. Laundering rate by payment type

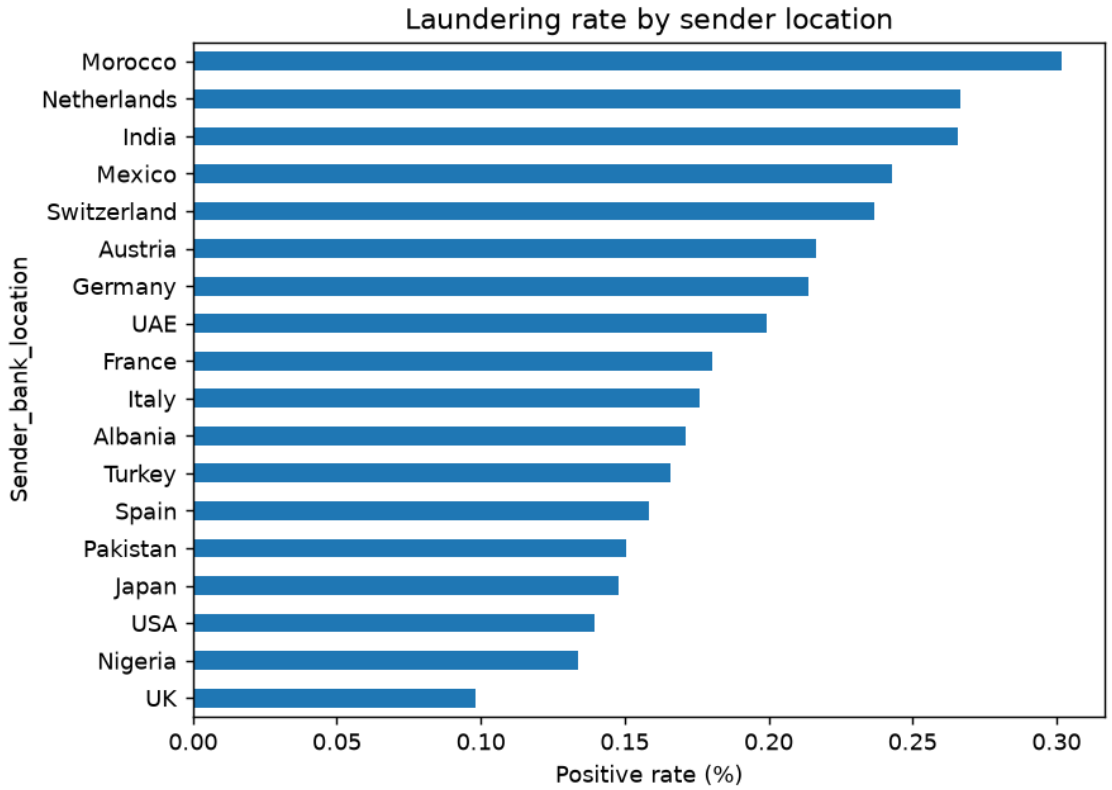


Figure 4. Laundering rate by sender bank location

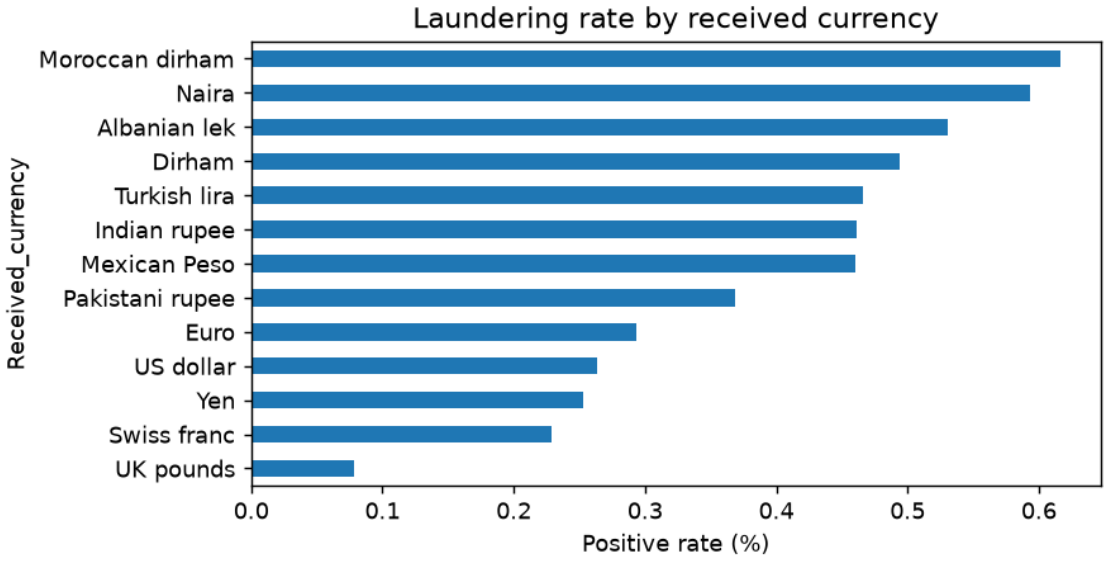


Figure 5. Laundering rate by received currency

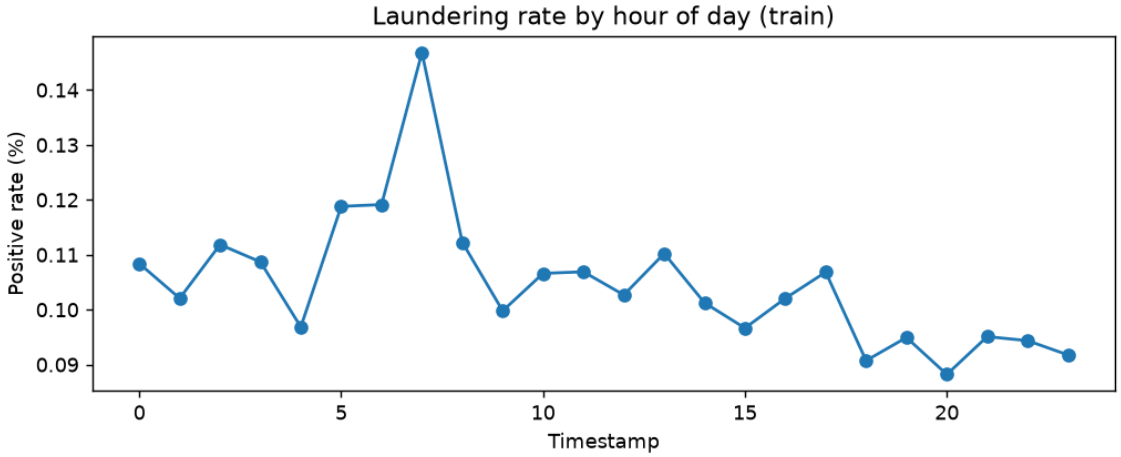


Figure 6. Laundering rate by hour of day

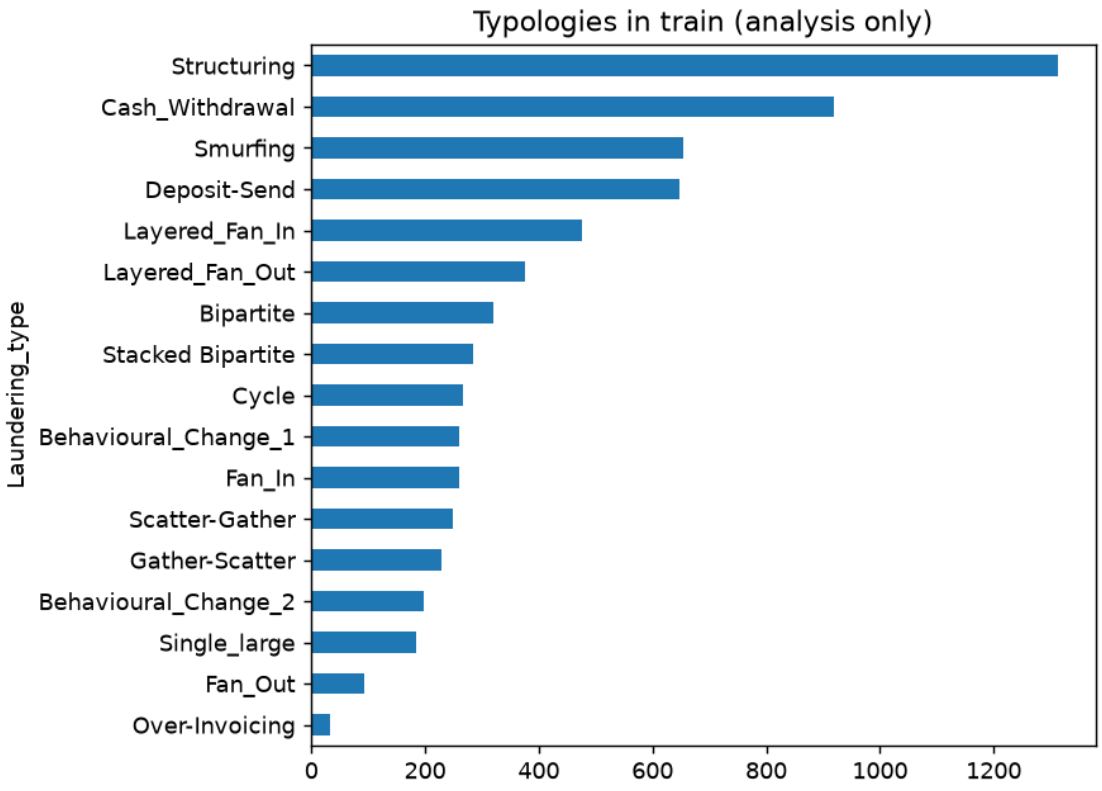


Figure 7. Laundering typologies in the training window (analysis only)

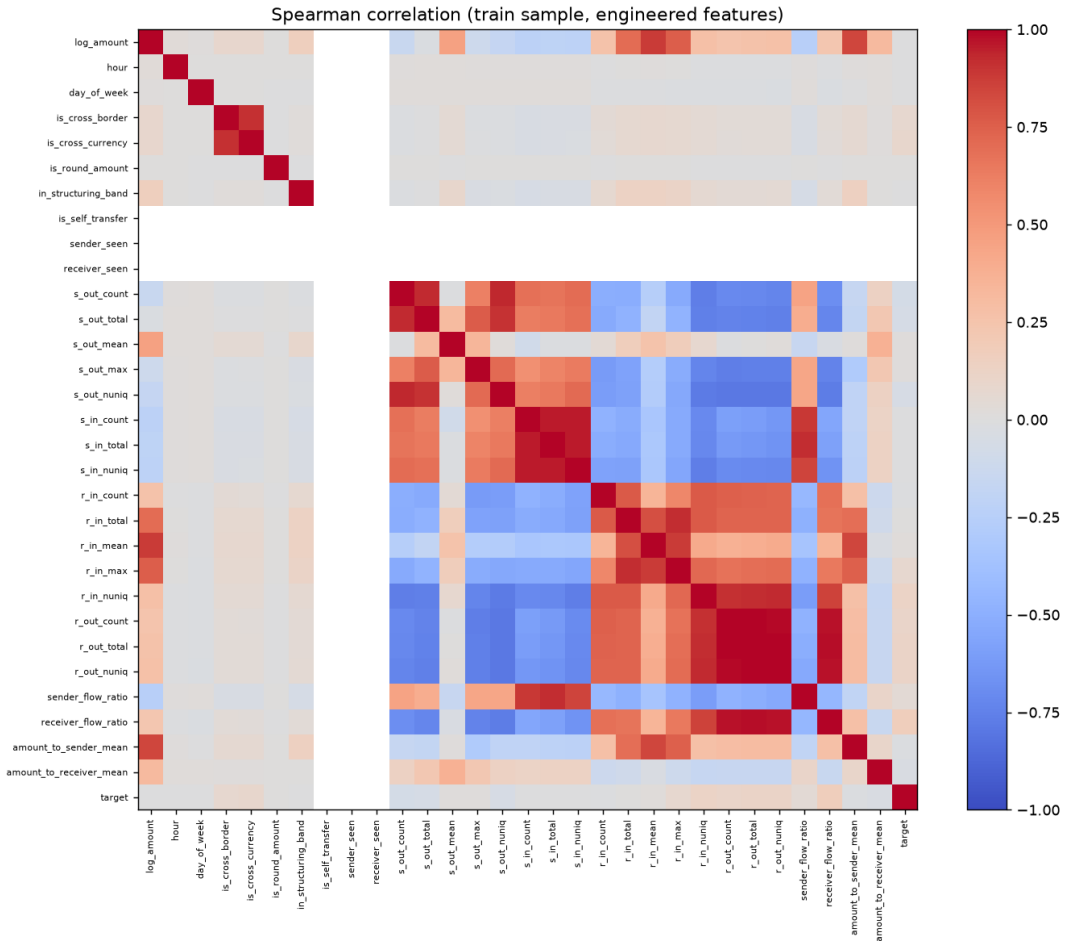


Figure 8. Spearman correlation of engineered features with the target

Table 5. Signal of engineered features: training versus validation

Feature	Univariate AUC (train)	Univariate AUC (validation)
receiver_flow_ratio	0.8642	0.7618
r_in_nuniq	0.7475	0.6833
r_out_nuniq	0.7414	0.7101
r_out_total	0.7376	0.7025
r_out_count	0.7258	0.6953
r_in_max	0.6807	0.6522
s_out_count	0.3378	0.4677
s_out_total	0.3555	0.4736

Insight: account-profile signals are weaker on validation than on training when profiles are computed over the whole training window. This motivated causal (lagged) profiles, described in the next section.

4. Feature engineering, imbalance handling and validation strategy

Feature engineering

- Row features: log amount, hour, day of week, cross-border flag, cross-currency flag, round-amount flag, 9,000-10,000 band flag, self-transfer flag.
- Categorical features: payment type, payment and received currency, sender and receiver bank location (unseen values map to a reserved level).
- Account profiles for sender and receiver: transaction counts, totals, mean and maximum amounts, number of distinct counterparties, and in/out flow ratios.
- Causal profiles: for every 7-day block the profiles are built only from the preceding 28 days, so no row sees its own or future activity and every row sees a history of the same length. The first 28 days are warm-up and excluded from training.
- Account identifiers are never used as raw features.

The feature builder produces 35 model inputs.

Validation strategy

A chronological 70/15/15 split is used. Training is fitted on the training window, hyper-parameters and the decision threshold are chosen on the validation window, and the test window is scored once with thresholds fixed beforehand. Resampling is applied inside the training fit only. Cross-validated average precision was used during tuning but is optimistic here because folds share accounts and time, so model selection relies on the validation window.

Imbalance experiments

Table 6. Imbalance strategies compared on the validation window (default hyper-parameters)

Model	Strategy	PR-AUC	ROC-AUC	Best F1	Precision	Recall
Logistic Regression	none	0.1658	0.9148	0.2169	0.2374	0.1997
Logistic Regression	class_weight	0.1167	0.9246	0.1649	0.2289	0.1289
Logistic Regression	random_over	0.1540	0.9213	0.2023	0.2204	0.1870
Logistic Regression	random_under	0.1494	0.9211	0.2009	0.2027	0.1990
Decision Tree	none	0.3393	0.9120	0.4653	0.5956	0.3817
Decision Tree	class_weight	0.4796	0.9526	0.5134	0.4464	0.6041
Decision Tree	random_over	0.5587	0.9361	0.6288	0.8551	0.4972
Decision Tree	random_under	0.4380	0.9551	0.5564	0.6905	0.4660
Decision	smotenc	0.2561	0.9491	0.3932	0.3055	0.5517

Tree						
Random Forest	none	0.7741	0.9833	0.7727	0.8277	0.7245
Random Forest	class_weight	0.6771	0.9857	0.6540	0.6755	0.6339
Random Forest	random_over	0.7787	0.9837	0.7684	0.8015	0.7380
Random Forest	random_under	0.6977	0.9831	0.6804	0.7554	0.6190
Random Forest	smotenc	0.6678	0.9828	0.6410	0.6575	0.6254

Strategies: none; class weights; random oversampling of the minority class to a 1:10 ratio; random undersampling to 1:10; and SMOTENC (after undersampling) for tree models only, because it needs ordinal-coded categoricals. Precision, recall and F1 in this table use a validation-tuned threshold and are therefore optimistic.

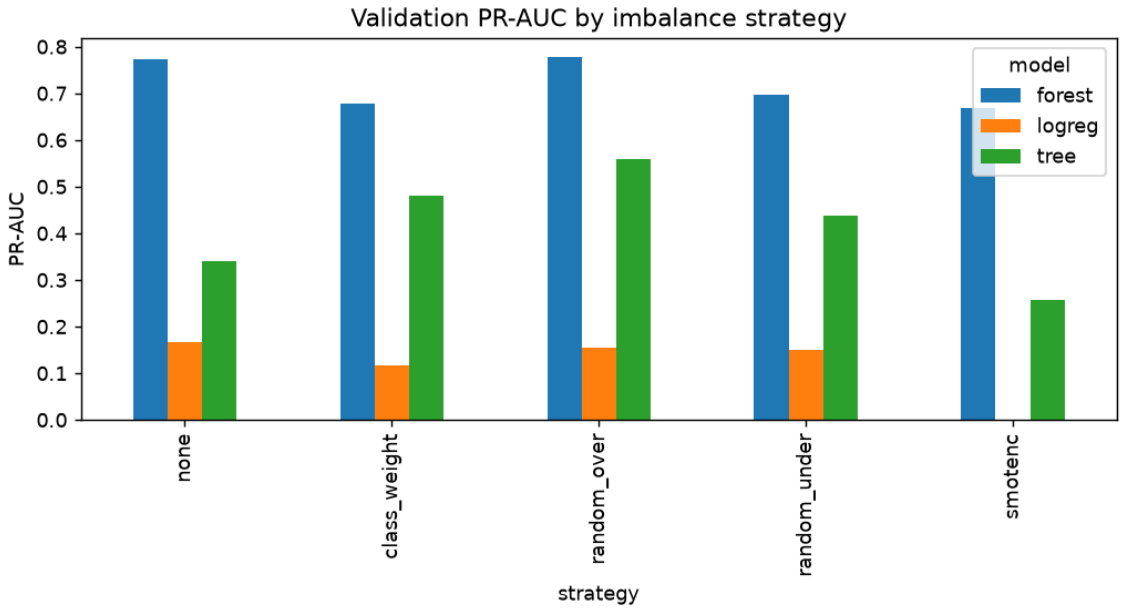


Figure 9. Validation PR-AUC by imbalance strategy and model

Table 7. Logistic Regression re-run with converged solver (max_iter=1000)

Strategy	**PR-AUC**	**ROC-AUC**
none	0.1658	0.9148
class_weight	0.1167	0.9246
random_over	0.1540	0.9213
random_under	0.1494	0.9211

5. Model building, tuning, evaluation and selection

Table 8. Randomised search (3-fold stratified CV, average precision)

Model	Strategy	Best hyper-parameters	CV average precision
Logistic Regression	none	{"C": 4.5705630998014515}	0.5804
Decision Tree	random_over	{"min_samples_leaf": 50, "max_depth": 16, "criterion": "entropy"}	0.8618
Random Forest	random_over	{"n_estimators": 60, "min_samples_leaf": 5, "max_features": 0.3, "max_depth": 20}	0.9255

CV scores are computed on a sample with a higher positive share than the real data and are not comparable with validation or test PR-AUC.

Table 9. Validation metrics at the threshold that maximises validation F1

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Positives
Logistic Regression	0.9986	0.2347	0.2011	0.2166	0.9153	0.1652	1,412
Decision Tree	0.9994	0.7070	0.7125	0.7097	0.9550	0.5658	1,412
Random Forest	0.9996	0.8181	0.7358	0.7748	0.9856	0.7808	1,412

Table 10. Test metrics at the threshold that maximises validation F1

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Positives
Logistic Regression	0.9984	0.3017	0.2357	0.2646	0.9207	0.2161	1,697
Decision Tree	0.9994	0.7704	0.7136	0.7409	0.9556	0.6168	1,697
Random Forest	0.9995	0.8474	0.7001	0.7667	0.9828	0.7973	1,697

Table 11. Test confusion matrices

Model	TP	FP	FN	TN	Alerts raised
Logistic Regression	400	926	1,297	1,423,105	1,326
Decision Tree	1,211	361	486	1,423,670	1,572

Random Forest	1,188	214	509	1,423,817	1,402

Operating point for higher recall (selected model, threshold 0.7456 chosen on validation for recall 0.80): test recall 0.780, precision 0.701, 1,889 alerts.

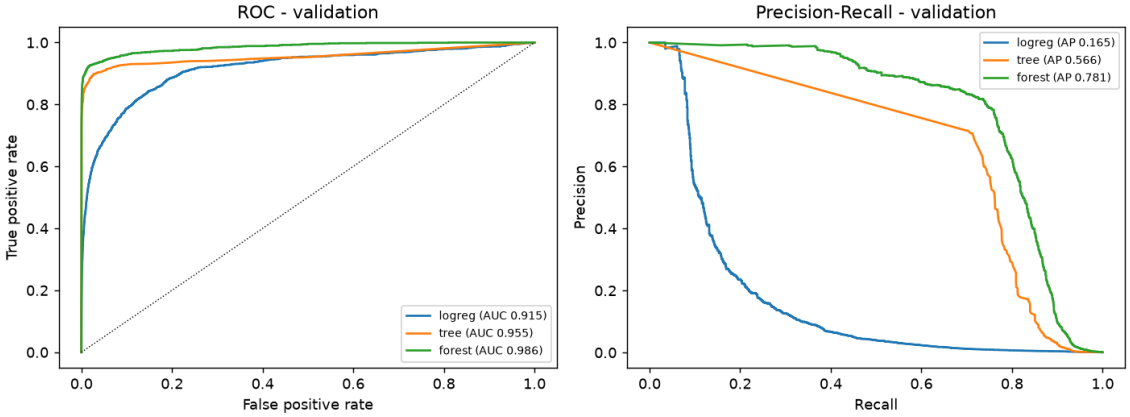


Figure 10. ROC and precision-recall curves (validation)

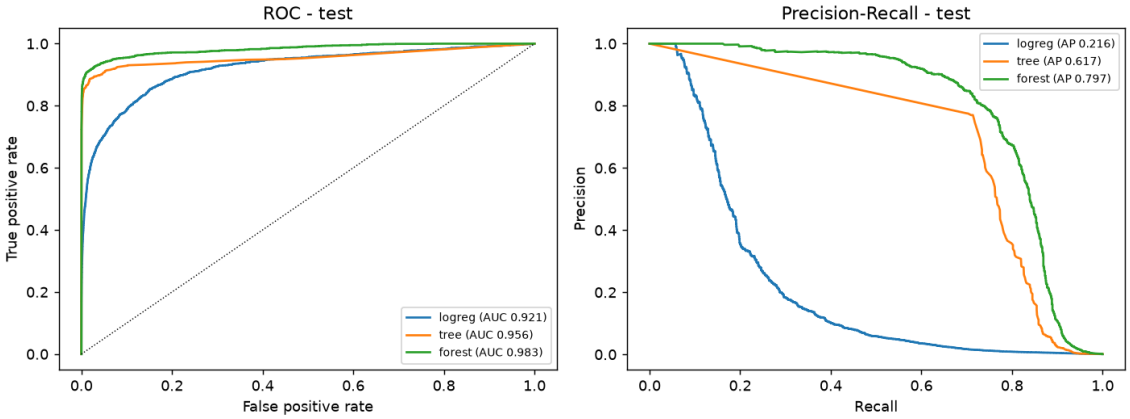


Figure 11. ROC and precision-recall curves (test)

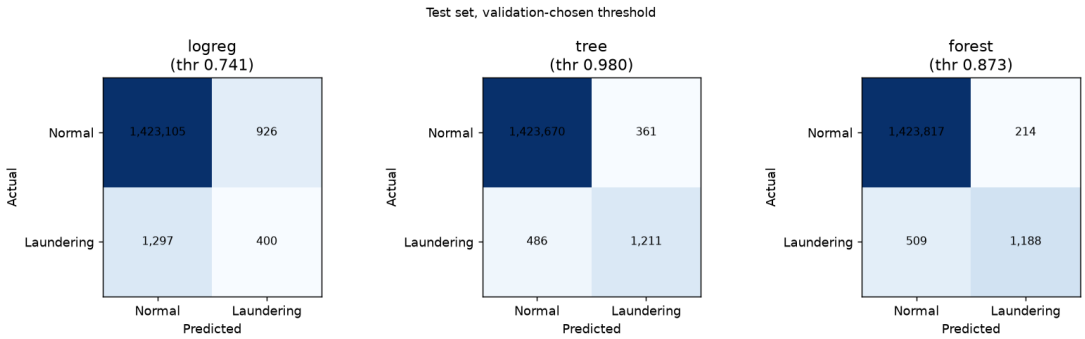


Figure 12. Confusion matrices on the test window

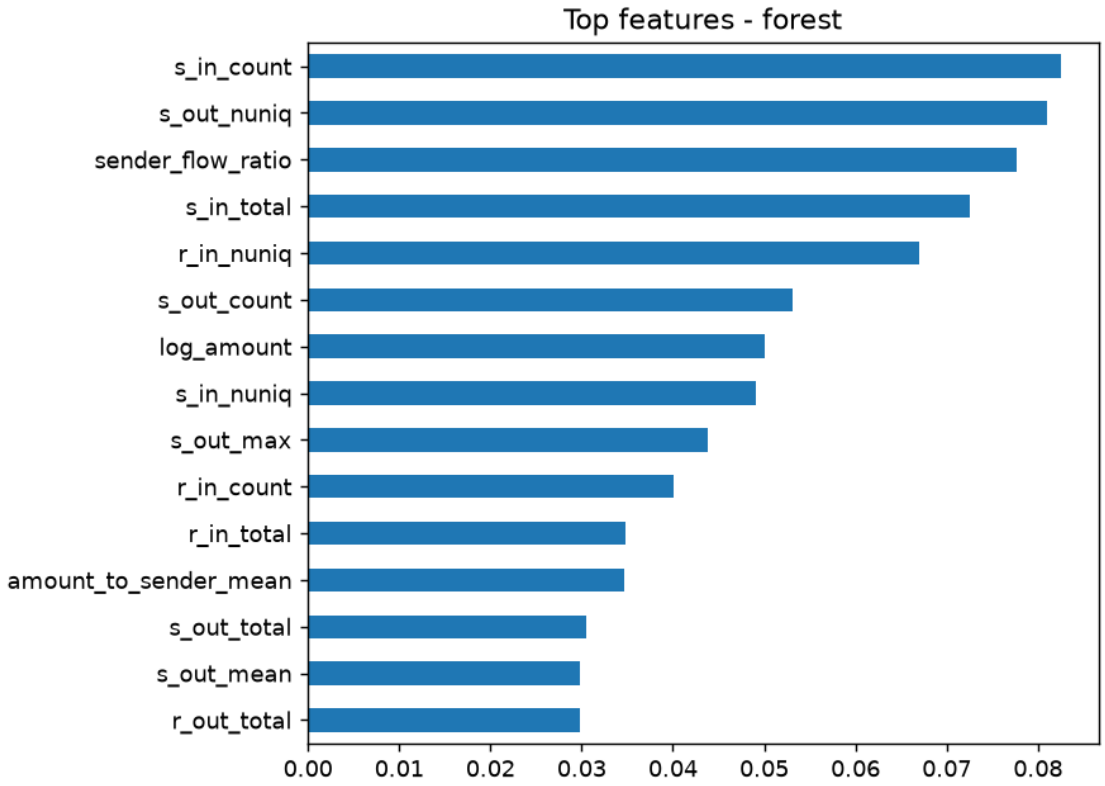


Figure 13. Feature importance of the selected model

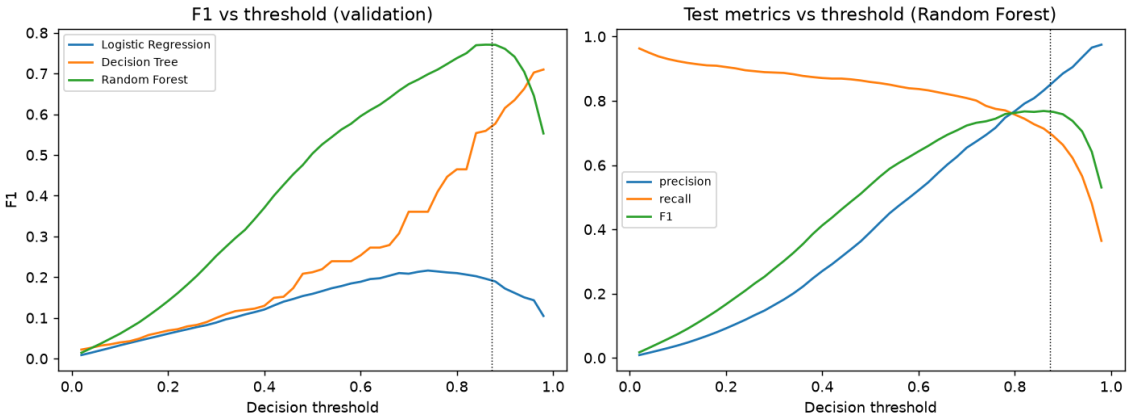


Figure 14. Precision, recall and F1 across decision thresholds

Selection: Random Forest has the highest validation PR-AUC (0.781). Accuracy is reported but not used for selection because a model that flags nothing already exceeds 99.8% accuracy on this data.

Table 12. Recall by laundering typology for the selected model (test)

Typology (analysis only)	Test cases	Recall
Bipartite	26	0.5000
Fan_In	36	0.5560
Cycle	49	0.5710
Over-Invoicing	7	0.5710
Gather-Scatter	82	0.5980
Single_large	34	0.6470
Layered_Fan_In	88	0.6480
Stacked Bipartite	128	0.6640
Smurfing	152	0.6640
Fan_Out	58	0.6720
Deposit-Send	163	0.6870
Structuring	363	0.7130
Scatter-Gather	46	0.7390
Layered_Fan_Out	100	0.7500
Cash_Withdrawal	241	0.7630
Behavioural_Change_1	62	0.8390
Behavioural_Change_2	62	0.8710

6. Model serialisation (.pkl) and validation

The deployable pipeline (feature builder with a frozen account-profile table, preprocessing, classifier) is saved with joblib as models/aml_pipeline.pkl, with the decision threshold in models/model_meta.json. Pickle files execute code when loaded: load only artifacts you built yourself.

Table 13. Serialisation and artifact validation

Check	Result
Reload prediction match (20,000 rows), max abs difference	0.0000
Reloaded predictions identical	True
Scoring time for the whole test window (s)	13.9000
Static profile ends at	2023-07-05 15:24:39
Artifact test PR-AUC (raw rows, static profile)	0.7645
Artifact test precision / recall / F1	0.808 / 0.681 / 0.739

The frozen profile becomes stale as time passes after its end date, which lowers performance slightly compared with refreshed profiles; a refresh job is part of the retraining plan.

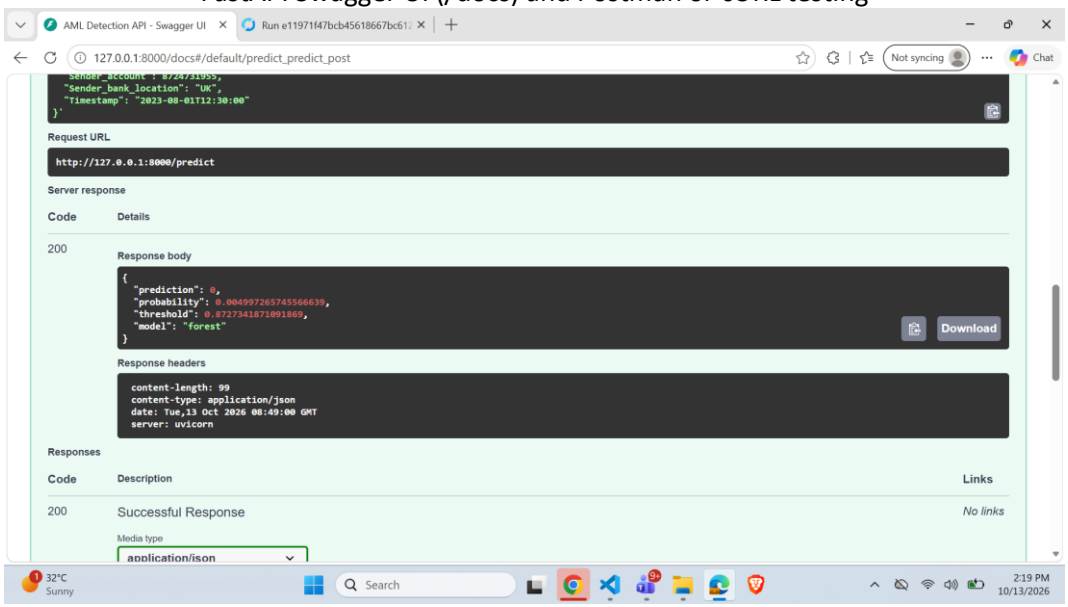
7. FastAPI design, testing and CI pipeline

Table 14. API endpoints

Endpoint	Purpose	Notes
GET /health	Service and model status	HTTP 503 when the model or the Azure backend is not configured
POST /predict	Score one transaction	Returns prediction, probability, threshold, model

- Request schema (Pydantic): Timestamp, Sender_account, Receiver_account, Amount (greater than 0, at most 1e9), Payment_type, Payment_currency, Received_currency, Sender_bank_location, Receiver_bank_location. Categories are restricted to the values seen in training; extra fields are rejected.
- Validation errors return HTTP 422 with location, message and type only; submitted values are never echoed or logged.
- Backends: local (loads the .pkl) or azure (forwards to the managed online endpoint using the key). Upstream failures map to HTTP 502 or 504 without leaking the key.
- The same inference code is used by the API, offline checks and the Azure scoring script, and tests assert identical probabilities.
- Automated tests (pytest and FastAPI TestClient) cover valid input, each missing field, wrong types, out-of-range values, unseen categories, extra fields, NaN, timezone-aware timestamps, model-load failure, corrupt model files, API versus local parity, and the Azure-backend error paths against a local stub.

FastAPI Swagger UI (/docs) and Postman or cURL testing



8. Azure MLflow implementation: tracking, registry, deployment, monitoring

Experiment tracking and registry

mlops/track_models.py retrains the three models with the tuned settings and logs, for each run, parameters, validation and test metrics (precision, recall, F1, ROC-AUC, PR-AUC, confusion counts), ROC/PR and confusion-matrix plots, a feature-importance plot, a data hash, the dependency lock file and the deployable model with its input signature. The best run by validation PR-AUC is registered as 'aml-detector' with version tags (stage, alias, threshold, selection rule, data hash).

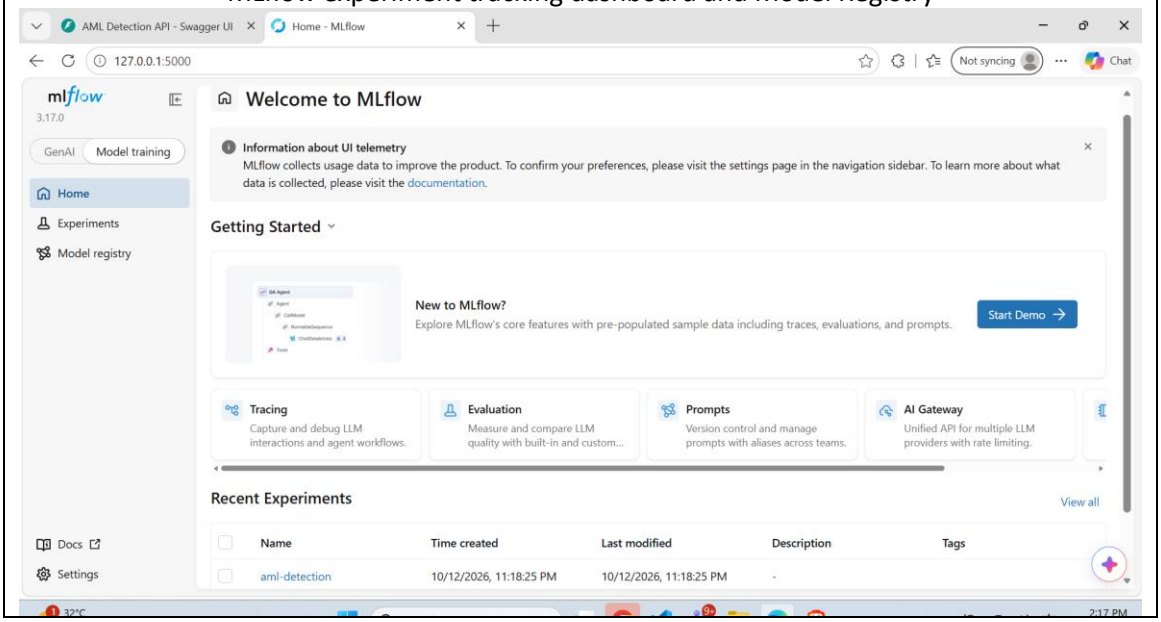
Table 15. MLflow run comparison (local tracking store)

Model	Validation PR-AUC	Test PR-AUC	Test F1	Difference vs earlier run
Random Forest	0.7808	0.7973	0.7667	0.0000
Decision Tree	0.5658	0.6168	0.7409	0.0000
Logistic Regression	0.1652	0.2161	0.2646	0.0000

Azure ML SDK v2 pipeline

mlops/azure/submit_pipeline.py (or the notebook notebooks/azureml_sdk2_pipeline.ipynb, run on the 'Python 3.10 - SDK v2' kernel) creates in the Azure ML workspace: a data asset (saml-d-raw), an environment (aml-train-env, Python 3.11.9), five command components (prepare, features, imbalance, tune, final), a pipeline job that connects them, and a registered model (aml-detector-scoring). Each step logs parameters, metrics, stepped precision/recall/F1 curves, and PNG and JSON artifacts through MLflow. The step scripts were run end to end locally with the job environment's pinned libraries and reproduced the reported model metrics exactly; the Azure run itself is not reported here.

MLflow experiment tracking dashboard and Model Registry



Deployment

Design: a managed online endpoint (key authentication) with a deployment named 'blue' on one Standard_DS2_v2 instance, a custom scoring script (mlops/azure/score.py) that reuses the same inference code, a conda environment with pinned dependencies, and Application Insights enabled. The scoring package and the Azure inference server were tested locally; no statement about the Azure deployment itself is made here.

Monitoring, retraining and safe rollout

- Monitoring: Application Insights for request volume, latency and errors; track the share of alerts per day and population drift of Amount, payment-type mix and cross-border share against the training window.
- Performance monitoring needs confirmed investigation outcomes; compare precision and recall on labelled cases each period.
- Retraining triggers: scheduled (for example monthly, to refresh account profiles) and event-based (drift beyond agreed limits or a fall in precision or recall).
- Safe rollout: deploy a candidate as 'green' beside 'blue', route a small traffic share, compare, then promote or roll back by moving traffic; delete the unused deployment to stop its cost.
- Cost: a managed deployment bills per hour while it exists. Delete the endpoint after testing (teardown script).

9. Results, limitations, ethical considerations and future work

Limitations

- The dataset is synthetic; results do not demonstrate real-world AML performance.
- Typologies such as bipartite, fan-in and cycle patterns are detected less well than others; see the per-typology table.
- The deployed profile table is a snapshot and goes stale; one transaction is scored without live account history.
- Training used a sampled negative pool because of compute limits; full-data training may change results.
- The model was trained on the training window only, not refit on training plus validation.

Ethical considerations

- False positives burden customers and investigators; thresholds should reflect investigator capacity, and alerts should trigger human review, not automatic action.
- Bias: location and currency features can proxy for nationality; fairness should be assessed on real data before use.
- Privacy: transaction and account data are sensitive; the API does not log request values, keys stay in environment variables or secret stores, and access should be restricted.
- Explainability: feature importance and alert reasons should be available to investigators and auditors.

Future work

- Gradient-boosted models and graph features for the weak typologies.
- Live account-profile store and drift-triggered retraining.
- Optional CI/CD to train, register and deploy to Azure with a service principal.

Selected model: Random Forest.

10. References

- SAML-D synthetic anti-money-laundering dataset (the source named in the project brief).
- scikit-learn, imbalanced-learn, pandas, FastAPI, Pydantic, pytest documentation.
- MLflow documentation (tracking, model registry); Azure Machine Learning documentation (managed online endpoints).

Appendix A: environment

Table 16. Library versions

Package	Version (this report run)
pandas	3.0.6
numpy	2.4.6
scikit-learn	1.9.1
imbalanced-learn	0.14.2
fastapi	0.143.0
pydantic	2.14.0
mlflow-skinny	3.17.0
python-docx	1.2.0

Appendix B: reproduction commands (PowerShell, project root, PYTHONPATH=src)

```
python -m aml.profile_data

python -m aml.eda

python -m aml.train features

python -m aml.train imbalance

python -m aml.train tune --cv-neg 120000

python -m aml.train final

python -m aml.verify artifact

python -m aml.verify logreg

python -m aml.verify auc

python -m mlops.track_models
```

```
python -m aml.report
```